

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

*I Pragadish R (192572030)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1.A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF

Answer:

Scenario: University Student & Course Database (Normalization up to 3NF)

Single Table:

Student_ID	Student_Name	Department	Course_ID	Course_Name	Faculty	Faculty_Phone
S101	Rahul	CSE	C101	DBMS	Kumar	9876543210
S101	Rahul	CSE	C102	Java	Priya	9876501234

Issues:

- Data redundancy (repeated student and faculty details)
- Update anomaly
- Insertion anomaly
- Deletion anomaly

1NF:

- Ensure each column has atomic (single) values.
- Remove repeating groups.

2NF:

Split into:

- **Student** (Student_ID, Student_Name, Department)
- **Course** (Course_ID, Course_Name, Faculty, Faculty_Phone)
- **Enrollment** (Student_ID, Course_ID)

3NF:

Remove transitive dependency by creating a Faculty table:

- **Student** (Student_ID, Student_Name, Department)
- **Course** (Course_ID, Course_Name, Faculty_ID)
- **Faculty** (Faculty_ID, Faculty_Name, Faculty_Phone)
- **Enrollment** (Student_ID, Course_ID)

Results:

The database is in **3NF**, reducing redundancy and eliminating update, insertion, and deletion anomalies.

2. A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach

Answer:

Scenario: Employee and Department Database

Tables

Employee

Emp_ID	Emp_Name
101	Arun
102	Priya

Employee_Department

Emp_ID	Dept_ID
101	D1
101	D2
102	D1

SQL Query:

```
SELECT Emp_ID
FROM Employee_Department
GROUP BY Emp_ID
HAVING COUNT(DISTINCT Dept_ID) > 1;
```

To display employee names:

```
SELECT E.Emp_ID, E.Emp_Name
FROM Employee E
JOIN Employee_Department ED
ON E.Emp_ID = ED.Emp_ID
GROUP BY E.Emp_ID, E.Emp_Name
HAVING COUNT(DISTINCT ED.Dept_ID) > 1;
```

3. Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

Answer:

Scenario: BCNF Decomposition

Relation:

R(Student_ID, Course_ID, Faculty)

Functional Dependencies:

- (Student_ID, Course_ID) → Faculty
- Faculty → Course_ID

Candidate Key:

- **(Student_ID, Course_ID)** is the candidate key because it determines all attributes.

BCNF Check:

- Faculty → Course_ID violates BCNF because **Faculty** is **not** a candidate key.

BCNF Decomposition:

Relation 1: Faculty_Course(Faculty, Course_ID)

Relation 2: Student_Faculty(Student_ID, Faculty)

Result:

The relation is now in **BCNF** because, in each decomposed relation, the determinant is a candidate key.

4. A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

Answer:

Scenario: Banking System – Views and Constraints

Create Table:

```
CREATE TABLE Account (  
    Acc_No INT PRIMARY KEY,  
    Customer_Name VARCHAR(50),  
    Balance DECIMAL(10,2) CHECK (Balance >= 0),  
    Branch VARCHAR(30)  
);
```

Create View (for Bank Staff):

```
CREATE VIEW Staff_View AS  
SELECT Acc_No, Customer_Name, Branch  
FROM Account;
```

Create View (for Manager):

```
CREATE VIEW Manager_View AS  
SELECT *  
FROM Account;
```

Constraints Used:

- **PRIMARY KEY** – Ensures unique account numbers.
- **CHECK (Balance >= 0)** – Prevents negative balances.
- **NOT NULL** (can be added for mandatory fields).
- **UNIQUE** (optional for fields like email or phone).

Result:

- **Views** restrict users to only the data they are authorized to see.
- **Constraints** maintain data accuracy, consistency, and integrity in the banking system.

5.A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.

Answer:

Scenario: Sales Database Normalization (Up to 3NF)

Unnormalized Table:

Order_ID	Customer_Name	Product	Salesperson	Salesperson_Phone
O101	Rahul	Laptop	Kumar	9876543210
O102	Rahul	Mouse	Kumar	9876543210

Problems:

- Redundant customer and salesperson data.
- Update, insertion, and deletion anomalies.

1NF:

- Ensure each field contains only a single value.

2NF:

Split into:

- **Customer**(Customer_ID, Customer_Name)
- **Order**(Order_ID, Customer_ID, Product, Salesperson_ID)

3NF:

Create a separate Salesperson table:

- **Customer**(Customer_ID, Customer_Name)
- **Salesperson**(Salesperson_ID, Salesperson_Name, Phone)
- **Order**(Order_ID, Customer_ID, Product, Salesperson_ID)

Result:

The schema is in **3NF**, eliminating redundancy and preventing update, insertion, and deletion anomalies.

6. Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

Answer:

Scenario: Customers and Orders Database

Customer Table:

Customer(Customer_ID, Customer_Name)

Orders Table:

Orders(Order_ID, Customer_ID, Order_Date)

1. Using JOIN

```
SELECT C.Customer_ID, C.Customer_Name, COUNT(O.Order_ID) AS Total_Orders
FROM Customer C
```

```

JOIN Orders O
ON C.Customer_ID = O.Customer_ID
GROUP BY C.Customer_ID, C.Customer_Name
HAVING COUNT(O.Order_ID) > 2;

```

2. Using Subquery

```

SELECT Customer_ID, Customer_Name
FROM Customer
WHERE Customer_ID IN (
    SELECT Customer_ID
    FROM Orders
    GROUP BY Customer_ID
    HAVING COUNT(Order_ID) > 2
);

```

Output:

Shows **frequent customers** who have placed **more than 2 orders**.

Justification:

- **JOIN** combines customer and order details.
- **Subquery** filters customers based on order count.
- **GROUP BY** and **HAVING** identify frequent customers efficiently.

7. A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.

Answer:

Scenario: 4NF Decomposition

Relation

R(Student_ID, Hobby, Language)

Example:

Student_ID Hobby Language

S101	Cricket	English
S101	Cricket	Tamil
S101	Music	English

Student_ID Hobby Language

S101 Music Tamil

Multi-Valued Dependencies

- Student_ID $\twoheadrightarrow$ Hobby
- Student_ID $\twoheadrightarrow$ Language

Since hobbies and languages are independent, the relation violates **4NF**.

Decomposition into 4NF

Student_Hobby

Student_ID	Hobby
S101	Cricket
S101	Music

Student_Language

Student_ID	Language
S101	English
S101	Tamil

Result

The relation is now in **4NF** because each table contains only one independent multi-valued dependency, eliminating redundancy and preventing update, insertion, and deletion anomalies.

8. A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest

Answer:

Scenario: Improper Constraints in Database

Problems

- Duplicate IDs
- NULL values in mandatory fields
- Negative salary or age
- Invalid foreign key references

Suggested Constraints

```
CREATE TABLE Employee (
```

```
    Emp_ID INT PRIMARY KEY,
```

```
    Emp_Name VARCHAR(50) NOT NULL,
```


Age INT CHECK (Age >= 18),
Salary DECIMAL(10,2) CHECK (Salary > 0),
Dept_ID INT,
FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);

Result

Using **PRIMARY KEY**, **NOT NULL**, **CHECK**, and **FOREIGN KEY** constraints ensures data integrity, prevents invalid or inconsistent data, and maintains relationships between tables.

9. For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.

Answer:

Scenario: 5NF Decomposition

Relation

R(Supplier, Part, Project)

Example:

<u>Supplier</u>	<u>Part</u>	<u>Project</u>
S1	P1	J1
S1	P2	J1
S2	P1	J2

Join Dependency

The relation has a **join dependency** because supplier–part, supplier–project, and part–project relationships are stored together, causing redundancy.

Decomposition into 5NF

Supplier Part

<u>Supplier</u>	<u>Part</u>
S1	P1
S1	P2
S2	P1

Supplier Project

<u>Supplier</u>	<u>Project</u>
S1	J1
S2	J2

Part Project

<u>Part</u>	<u>Project</u>
P1	J1
P2	J1
P1	J2

Result

The schema is now in **5NF**, eliminating redundancy caused by join dependencies. The original relation can be reconstructed by joining the three decomposed tables without any loss of information.

10. A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

Answer:

Scenario: SQL Query Optimization

Inefficient Query

```
SELECT *
FROM Employee
WHERE Dept_ID IN (
  SELECT Dept_ID
  FROM Department
  WHERE Dept_Name = 'Sales'
);
```

Optimized Query (Using JOIN)

```
SELECT E.*
FROM Employee E
JOIN Department D
ON E.Dept_ID = D.Dept_ID
WHERE D.Dept_Name = 'Sales';
```

Indexing

```
CREATE INDEX idx_deptid
```

ON Employee(Dept ID);

Result

- **JOIN** executes faster than nested subqueries in many cases.
- **Indexing** speeds up searching and retrieval.
- Query restructuring reduces execution time and improves overall database performance.